NIM – NAMA :

- 2802507562 – Andreas Tri Dewa Putra
- 2802512865 – Dhammika Kumara
- 2802512530 – Limanto Winata HM

Kelompok : Ga ADA / Group-4

Kelas : LF-01

Mata Kuliah : Algorithm Design & Methods

Soal : AOL

A.Problem E Unsolved

SOAL:



international collegiate programming contest
INDONESIA NATIONAL CONTEST
INC 2025



Problem E

Xordition Robot

You have a robot that contains N modules, numbered from 1 to N. Each module accepts an integer and outputs an integer. The output of module i becomes the input of module $i+1$ (for $1 \leq i \leq N-1$).

The specification of module i is either:

- `+ k`: given an integer x $(0 \leq x < 16)$, the module outputs $(x + k) \bmod 16$; or
- `x k`: given an integer x $(0 \leq x < 16)$, the module outputs $x \oplus k$, where $\oplus$ represents the bitwise XOR operator.

There are Q replacements, and the j-th is of the form:

- `i t k`: replace module i to a module with specification `t k`, where t is either `+` or `x`.

Each time a replacement is done, find the output of module N when module 1 is given an input 0.

Input

The first line contains two integers N and Q $(1 \leq N, Q \leq 200\ 000)$. Each of the next N lines contains a character of either `+` or `x` followed by an integer k $(0 \leq k < 16)$ representing the module.

The next Q lines contains an integer i $(1 \leq i \leq N)$, followed by a character `+` or `x`, and finally an integer k $(0 \leq k < 16)$, meaning that you have to replace module i to the specified module.

Output

Output Q lines, each containing the output of module N, after each replacement, when given an input 0 to module 1.

Sample Input 1	Sample Output 1
4 2 + 3 x 5 x 9 + 15 2 + 8 1 x 10	1 10

Kode yang digunakan saat ICPC:

```c
#include <stdio.h>

int main(){
	int modules, replacements;
	scanf("%d %d", &modules, &replacements);
	char operations[modules+1];
	int num[modules];
	for(int i=0; i<modules; i++){
		scanf("%s %d\n", &operations[i], &num[i]);
	}
	for(int i=0; i<replacements; i++){
		int number=0;
		int changeIdx, changeNum;
		char changeOp[2];

		scanf("%d %s %d", &changeIdx, changeOp, &changeNum);
		changeIdx -= 1;
		operations[changeIdx] = changeOp[0];
		num[changeIdx] = changeNum;
		for(int j=0; j<modules; j++){
			if(operations[j] == '+') number = (number + num[j]) % 16;
			else if(operations[j] == 'x') number = (number ^ num[j]);
		}
		printf("%d\n", number);
	}
	return 0;
}
```

Kode ini menemukan permasalahan yang berupa Time Limit Exceeded, hal tersebut berarti kode ini tidak efisien dalam sisi waktu. Kode tersebut mempunyai Time Complexity O(n*m) dengan n berupa nilai dari variabel replacements dan m berupa nilai dari variabel modules.

Setelah mencari solusi yang dapat menyelesaikan Problem E ini, kita menemukan Segment Tree cocok untuk permasalahan ini. Segment Tree adalah Data Structure yang dapat melakukan query dan mengubah interval atau segment dalam suatu array secara efisien.

SOLUSI:

```c
#include <stdio.h>
#include <math.h>
```

```
struct Modul {
    char tipe;
    int nilai;
};

Modul data_modul[200005];
int cache_grup[450][16];

int N, Q;
int panjang_grup;

int jalankan_operasi(int input, char tipe, int nilai) {
    if (tipe == '+') {
        return (input + nilai) % 16;
    } else {
        return (input ^ nilai);
    }
}

void perbarui_grup(int id_grup) {
    int mulai = id_grup * panjang_grup;
    int akhir = (id_grup + 1) * panjang_grup;

    if (akhir > N) {
        akhir = N;
    }

    for (int input_awal = 0; input_awal < 16; input_awal++) {
        int hasil = input_awal;
        for (int i = mulai; i < akhir; i++) {
            hasil = jalankan_operasi(hasil, data_modul[i].tipe,
data_modul[i].nilai);
        }
```

```
        cache_grup[id_grup][input_awal] = hasil;
    }
}

int main() {
    if (scanf("%d %d", &N, &Q) != 2) return 1;

    panjang_grup = (int)sqrt(N);
    if (panjang_grup == 0) {
        panjang_grup = 1;
    }

    for (int i = 0; i < N; i++) {
        if (scanf(" %c %d", &data_modul[i].tipe, &data_modul[i].nilai)
!= 2) return 1;
    }

    int total_grup = (N + panjang_grup - 1) / panjang_grup;

    for (int g = 0; g < total_grup; g++) {
        perbarui_grup(g);
    }

    for (int i = 0; i < Q; i++) {
        int idx_1based;
        char op_baru;
        int val_baru;

        if (scanf("%d %c %d", &idx_1based, &op_baru, &val_baru) != 3)
return 1;

int idx_0based = idx_1based - 1;
data_modul[idx_0based].tipe = op_baru;
```

```
data_modul[idx_0based].nilai = val_baru;
int id_grup_yang_diubah = idx_0based / panjang_grup;
perbarui_grup(id_grup_yang_diubah);
int jawaban_akhir = 0;
for (int g = 0; g < total_grup; g++) {
jawaban_akhir = cache_grup[g][jawaban_akhir];
}
printf("%d\n", jawaban_akhir);
}
return 0;
}
```

ANALISA:

Solusi untuk masalah Condition Robot menggunakan Segment Tree untuk memastikan efisiensi tinggi, mencapai kompleksitas O(N + Q log N), yang ideal untuk batasan N dan Q yang besar. Efektivitasnya berasal dari domain input/output yang sangat kecil (Z16), di mana setiap urutan modul dimodelkan sebagai fungsi f: Z16 → Z16 (disimpan sebagai array 16 elemen). Struktur pohon segmen ini memanfaatkan sifat asosiatif dari komposisi fungsi (fR ◦ fL) untuk menggabungkan dua rentang dalam waktu konstan O(16). Ketika modul diganti, hanya node daun yang diperbarui, dan perubahan ini dikomposisikan ulang ke atas pohon dalam O(log N) langkah. Sehingga , node akar Segment Tree secara terus menerus menyimpan fungsi total dari Modul 1 hingga N, memungkinkan pengambilan output akhir (untuk input 0) langsung tanpa memerlukan proses lagi.

Bukti:

